The Rules Are 7% of the Program

Josh · Secure Sovereign

Published October 30, 2026

A line count of Bitcoin Core at 11 May 2026

Commit `10ca73c`. The checkout is 2,407 commits behind origin/master as of 30 October 2026. Current-tree figures are tokei 14 code lines. History figures are git numstat on non-merge commits.

The consensus rules are **10,055** code lines. The implementation they sit in is **145,534**. The code that applies those rules, stores the result, or chooses what to relay and mine is **18,965**, about 1.9 times the rules. The other **116,514** lines are the wallet, the GUI, RPC, P2P transport, and the rest of the node.

A wrong rule forks the network. A policy disagreement does not. The two live in the same program, and the rules are the thin slice. That is the measurement behind *The Vertical Layer Problem* and *Why Bitcoin Needs a Specification That Humans Can Read*.

Contents

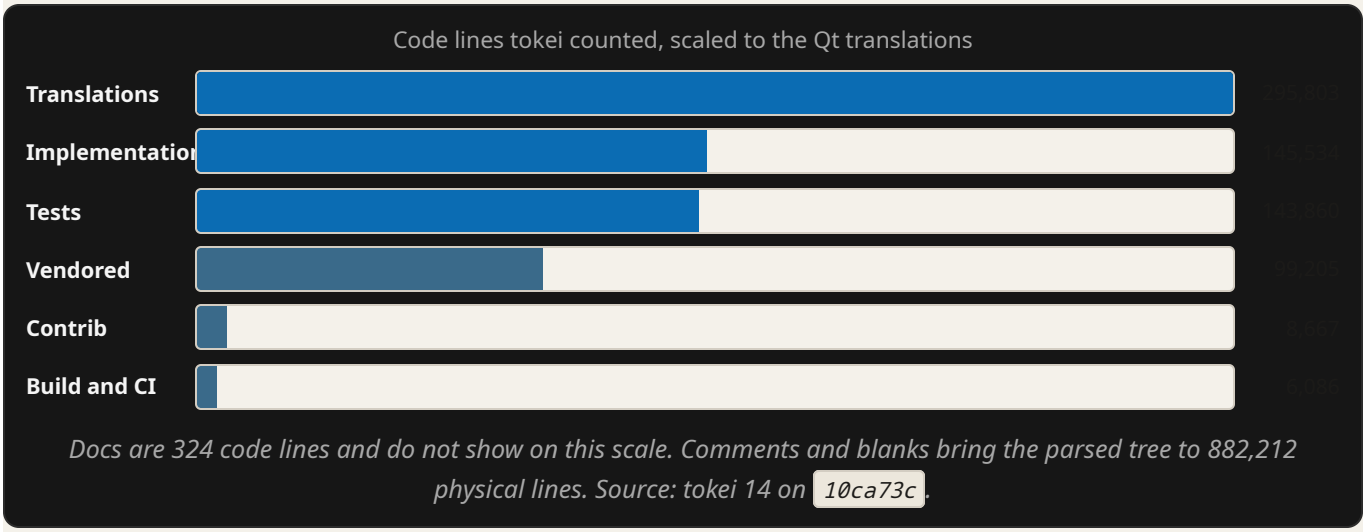
- I. What was counted
- II. The tree
- III. The implementation
- IV. The rules
- V. What sits beside the rules
- VI. Tests
- VII. Bundled code
- VIII. Who added the lines
- IX. Who last touched what remains
- X. Patch debt
- XI. What the count is for
- XII. Method

I. What was counted

Tokei parsed **2,632** files and **699,479** code lines. Every one of those files is in exactly one bucket below. Git tracks 2,949 files. The 317 tokei skipped are 206 text files (32,319 physical lines, mostly Qt forms, CI glue, and licenses) and 111 binary files (4.95 MB). They are not in the code totals.

The headline is the implementation, not the 699,479. Qt locale files are 295,803 of those lines and are translations, counted by tokei as TypeScript. Vendored libraries are another 99,205. First-party code, implementation plus tests plus contrib, build, and docs, is **304,471**.

II. The tree



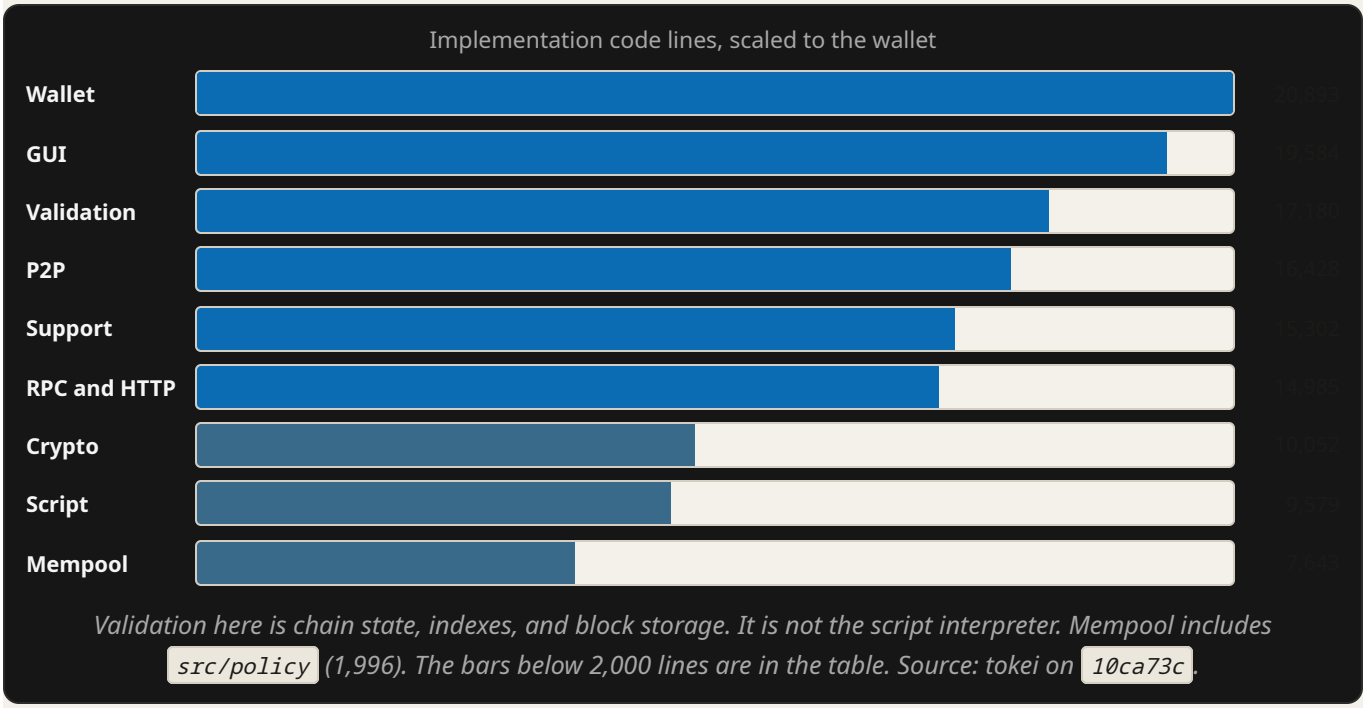
Bucket	Code	Comments	Blanks	Files
Qt translations	295,803	3	215	102
Implementation	145,534	35,398	24,804	744
Tests	143,860	21,810	25,600	882
Vendored	99,205	17,173	11,720	514
Contrib	8,667	4,213	1,494	69
Build and CI	6,086	777	949	105
Docs	324	30,516	8,061	216

Tests are 49.7% of implementation-plus-tests. Drop the JSON, hex, and HTML fixtures and test code is 131,905, which is 47.5% of implementation plus test code. The program carries about as much test code as implementation code.

First-party languages, the 304,471: C++ 172,095, Python 65,652, C and C++ headers 42,356, JSON 12,059, build files 9,819, Rust 1,186, other 1,304. Implementation bodies are 73.8% C++ (107,383) and 25.6% headers (37,312). The other 839 lines are neither.

III. The implementation

Seven hundred forty-four files. The median file is 64.5 code lines. Twenty-nine files of 1,000 lines or more hold 36.8% of the implementation. Fifty-four files hold half of it.



Validation here is chain state, indexes, and block storage. It is not the script interpreter. Mempool includes `src/policy` *(1,996). The bars below 2,000 lines are in the table. Source: tokei on* `10ca73c`.

Also in the 145,534	Code
Executables	2,829
Peer address seeds (`chainparamsseeds.h`)	2,651
Init	2,241
Node interfaces	1,751
Mining templates	824
Addresses	816
Logging	801
Interfaces	732
ZMQ	641

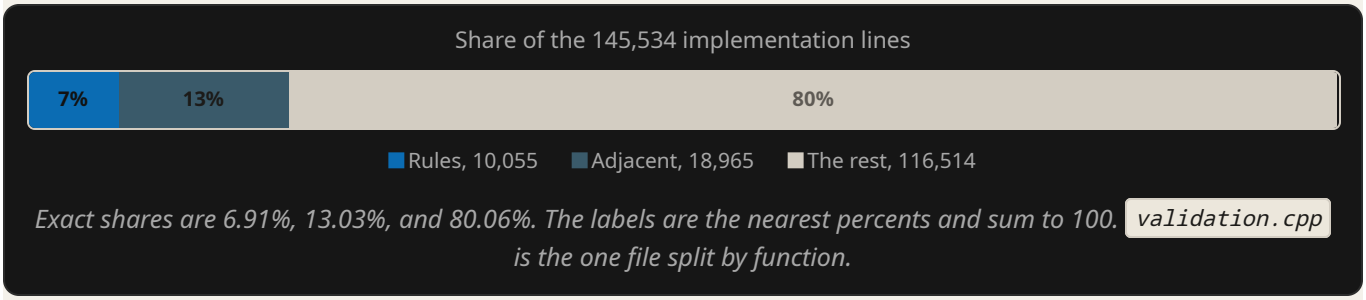
Also in the 145,534	Code
IPC	602

The largest files are `validation.cpp` (4,230), `net_processing.cpp` (3,989), `wallet.cpp` (3,451), `net.cpp` (3,174), `rpc/blockchain.cpp` (3,148), `chainparamsseeds.h` (2,651), `script/descriptor.cpp` (2,430), and `txgraph.cpp` (2,350).

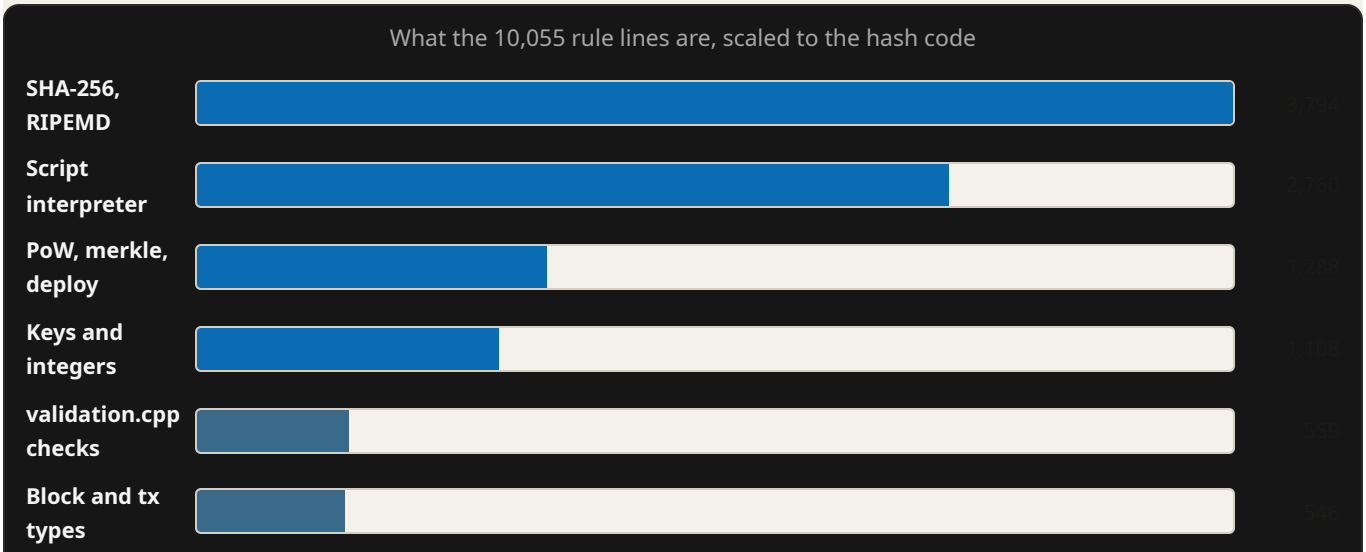
`src/node` is not all P2P. Block storage and chainstate in that directory are in the validation bar. P2P is the network files only.

IV. The rules

Consensus here means the code that decides whether a block or a transaction is valid. The bundled signature library is outside this split. It is counted in section VII.



Exact shares are 6.91%, 13.03%, and 80.06%. The labels are the nearest percents and sum to 100. `validation.cpp` *is the one file split by function.*



The hash bar is SHA-256, including the CPU-specific files, plus RIPEMD-160 and the hash wrappers. SHA-1, SHA-3, and SHA-512 are in the rest. The interpreter itself is 1,575 lines. The 2,760 includes its header, script serialization, and the script error codes.

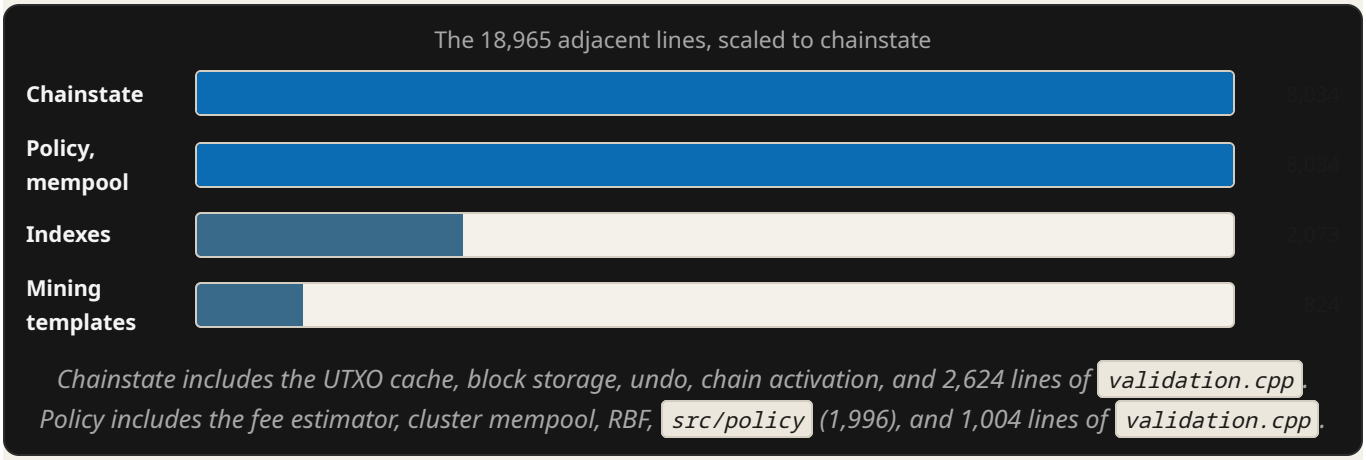
Core's exported library, `libbitcoinconsensus`, is **4,696** of these rule lines: the interpreter, script, merkle, transaction checks, hashes, and the basic types. The other **5,359** rule lines are outside it. **3,566** of those are the SHA-256 and RIPEMD-160 implementations. The other **1,793** are proof of work, versionbits, deployments, signet, transaction input checks, the 559 lines in `validation.cpp`, and a handful of headers.

The kernel library is a different list. Its own comment says the list will shrink. The named sources still compile policy and mempool, so the kernel target is not this boundary. `bitcoin_crypto` sums to 4,048 and includes `cleanse.cpp` and `lockedpool.cpp`.

`src/script/solver.h` says the solver is used by policy and the wallet, and not by consensus. Descriptors and miniscript are in the 116,514.

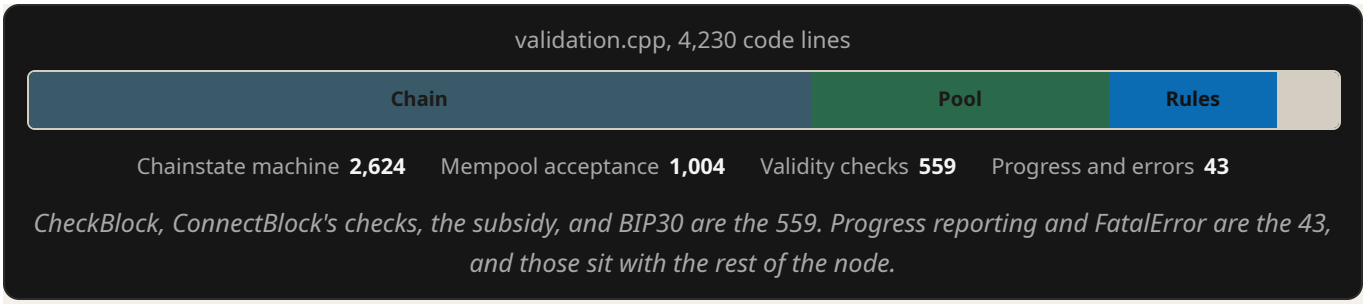
V. What sits beside the rules

Adjacent means the code that applies the rules, stores the result, or chooses what to relay and mine. An index can be wrong without making a block invalid. A policy limit can change without forking.



validation.cpp is mostly the machine around the checks

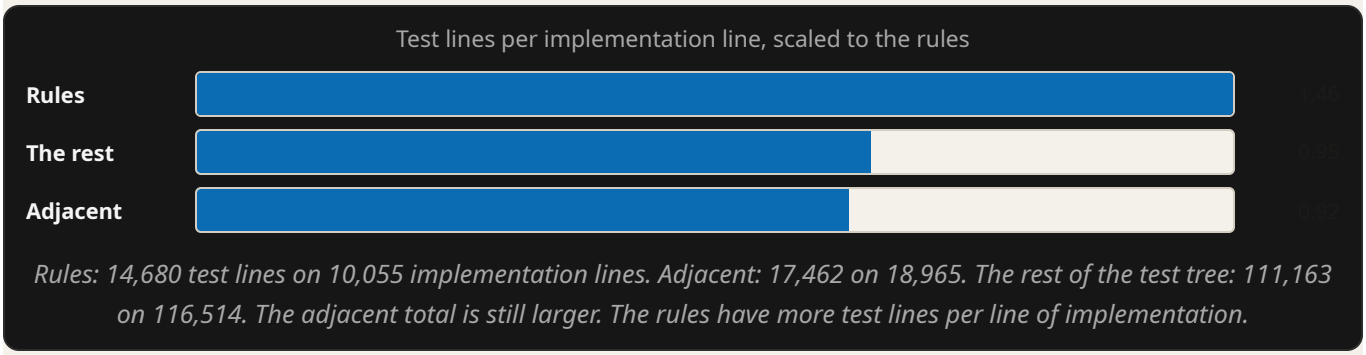
The file is 4,230 code lines. The split is at function boundaries, comment-stripped, then scaled so the parts still sum to tokei's figure. ConnectBlock's checks stay with the rules. The undo write at the end of that function stays with chainstate. MemPoolAccept stays with policy.



VI. Tests

Tests are 143,860 code lines: functional Python 58,210, unit 39,050, fuzz 20,665, fixtures 11,955, bench 6,405, wallet 3,141, lint 1,848, GUI 1,150, CI 854, harness 378, IPC 204.

A filename cut that stops at unit tests understates the functional suite. The assignment below puts every test file in one bucket. Rule tests are the named unit and fuzz files, further unit and fuzz of the rule types, the script and transaction vectors, and the functional tests of validity rules, including `p2p_segwit.py` (1,372), `feature_taproot.py` (1,350), and `feature_block.py` (926). Adjacent tests keep the mempool and policy names and add mining, assumeutxo, pruning, indexes, fee estimation, and package RPC. Benchmarks that share those names, 555 lines, stay out of the ratio.



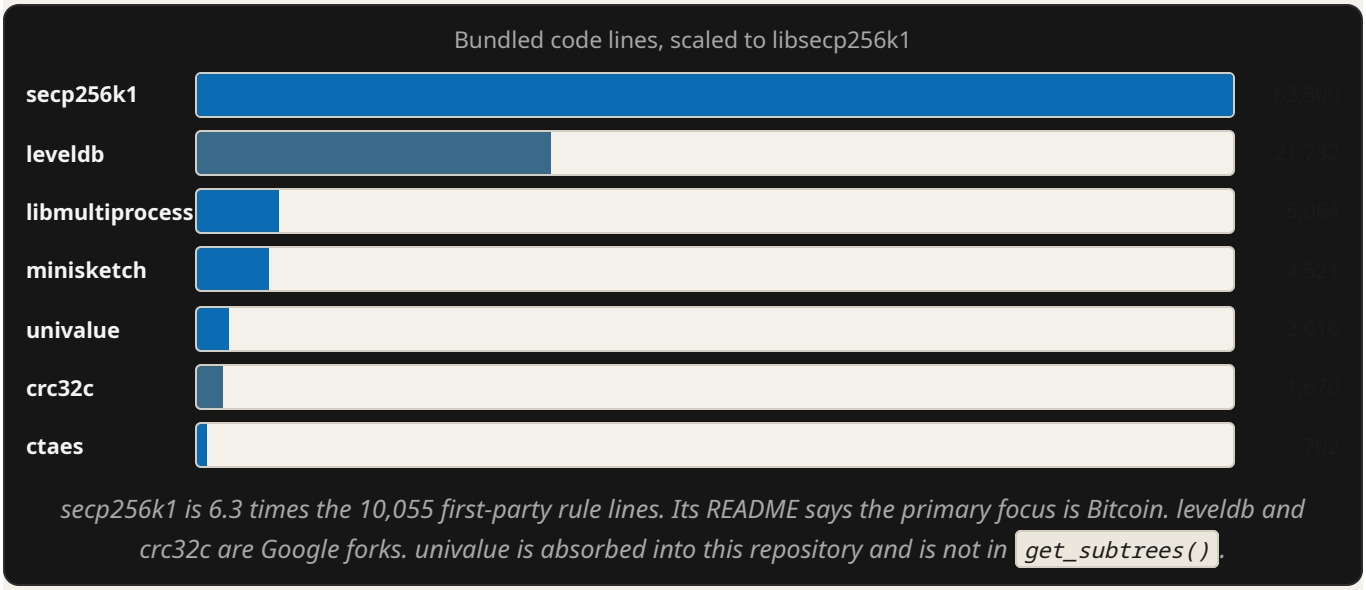
What was counted	Code lines	Bucket
Named rule unit and fuzz	4,440	Rules
Integer, hash, script, and block fuzz	1,346	Rules
Functional tests of validity rules	5,337	Rules
`script_tests.json`, tx vectors, `invalid_txs.py`	3,557	Rules
Mempool, policy, cluster, RBF, package names	8,022	Adjacent
Mining, assumeutxo, pruning, indexes, package RPC	6,472	Adjacent
Coins, miner, blockfilter, chainstate unit and fuzz	2,968	Adjacent

What was counted	Code lines	Bucket
Everything else in the test tree	111,163	The rest

The 3,557 vector lines are `script_tests.json` (2,591), `tx_valid.json` (426), `tx_invalid.json` (313), and `invalid_txs.py` (227). The rest row includes wallet, GUI, P2P transport, lint, and CI. A test that calls the interpreter under a name this list does not use is in that row.

VII. Bundled code

Vendored trees sum to 99,205 code lines. **75,803** (76%) are maintained as Bitcoin Core projects. **23,402** (24%) are forks of Google libraries. The patch size of those forks against Google's master was not measured.



secp256k1 is 6.3 times the 10,055 first-party rule lines. Its README says the primary focus is Bitcoin. leveldb and crc32c are Google forks. univalue is absorbed into this repository and is not in `get_subtrees()`.

Fetched at build time, and absent from the checkout, so they have no line count here: Qt 6.8.3, Boost 1.90.0, libevent 2.1.12-stable, SQLite 3.50.4, ZeroMQ 4.3.5, Cap'n Proto 1.4.0, qrencode 4.1.1, systemtap 5.3. Those are wider-community libraries. `src/bench/nanobench.h` is 2,380 lines of third-party nanobench sitting inside the test tree.

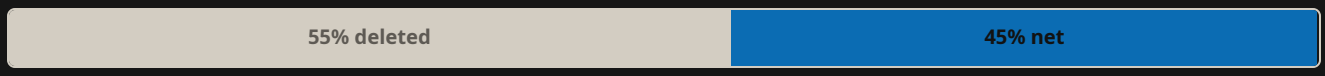
Public-key checks in `src/pubkey.cpp` call into secp256k1. The signature math is a Core project used by other software. It is not part of the node's own source layout, and it is larger than the first-party rules.

VIII. Who added the lines

History is git numstat on non-merge commits, extensions `.cpp`, `.h`, `.hpp`, `.c`, `.cc`, `.py`, and `.rs` only. Comments and blanks are included, so these are not tokei code lines. Vendored prefixes, Qt translations, and build scripts are excluded. Two bulk imports are dropped from the people table and kept in the year chart: sirius-m (46,010 added, 16 commits) and Vinnie Falco (25,695 added, 2 commits).

On that filter, **1,084,628** lines were added and **598,508** were deleted. Deleted over added is **55%**. Net is added minus deleted. It is not the lines still in the tree. That count is section IX.

Of the lines added to code files, through 11 May 2026



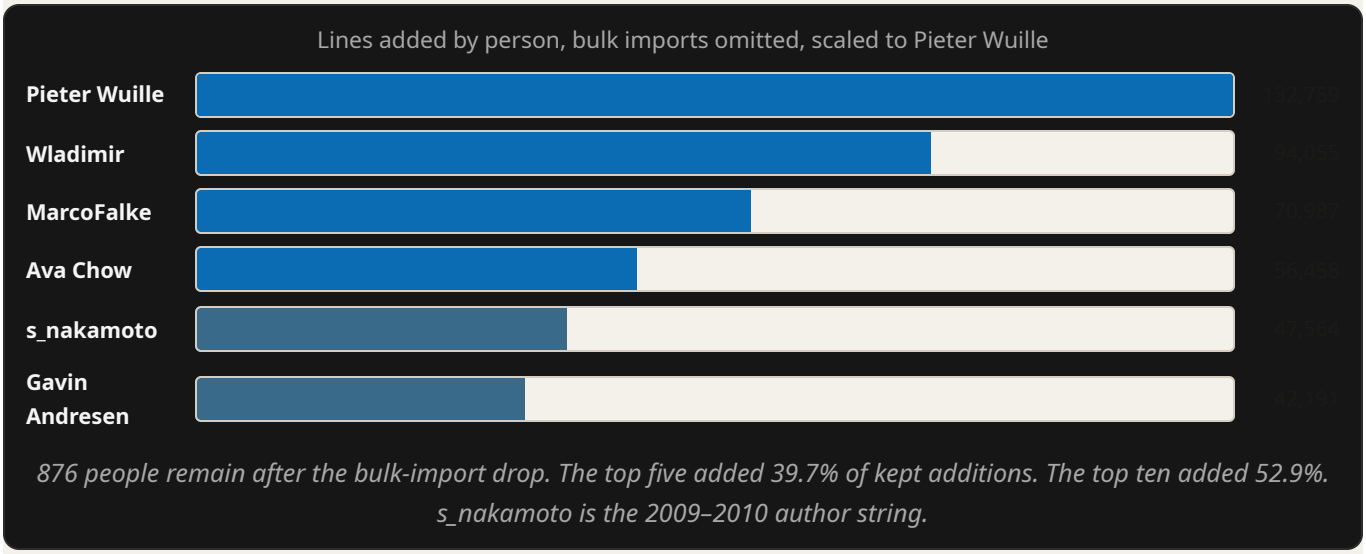
Added **1,084,628** Deleted **598,508** Net **486,120**

32,401 non-merge commits through this snapshot. 25,231 of them touched the filtered code files. The other 7,170 touched only build, docs, translations, or vendored paths.

Year	Added	Deleted	Code commits
2009	54,844	10,686	68
2010	65,289	62,104	218
2011	53,392	27,777	672
2012	37,038	21,369	958
2013	59,554	20,211	688
2014	64,249	36,468	1,273
2015	49,181	21,257	1,032
2016	57,430	28,521	1,400
2017	65,394	43,770	1,932
2018	56,407	34,265	1,648
2019	60,527	35,236	1,785
2020	91,782	47,157	2,606
2021	86,333	50,883	2,577
2022	69,544	33,989	2,134
2023	58,730	32,763	1,833

Year	Added	Deleted	Code commits
2024	76,404	44,704	1,982
2025	58,726	35,863	1,782
2026	19,804	11,485	643

2026 stops at 11 May. 2010's net is 3,185. The largest net year is 2020, at 44,625. 2009's net, 44,158, is close. Distinct author emails peaked at 152 in 2018 and fell to 79 in 2023. That email count double-counts a person who used two addresses in one year.



876 people remain after the bulk-import drop. The top five added 39.7% of kept additions. The top ten added 52.9%. s_nakamoto is the 2009–2010 author string.

Person	Added	Deleted	Net	Commits	Deleted share
Pieter Wuille	132,759	59,491	73,268	1,696	45%
Wladimir J. van der Laan	94,055	46,601	47,454	1,275	50%
MarcoFalke	70,987	59,916	11,071	2,791	84%
Ava Chow	56,458	38,452	18,006	1,197	68%
s_nakamoto	47,564	43,441	4,123	209	91%
Gavin Andresen	42,191	36,796	5,395	393	87%
John Newbery	24,882	26,142	−1,260	749	105%
Suhas Daftuar	24,723	8,580	16,143	438	35%
Russell Yanofsky	22,958	13,286	9,672	414	58%
glozow	19,444	5,951	13,493	390	31%
fanquake	17,545	17,104	441	580	97%

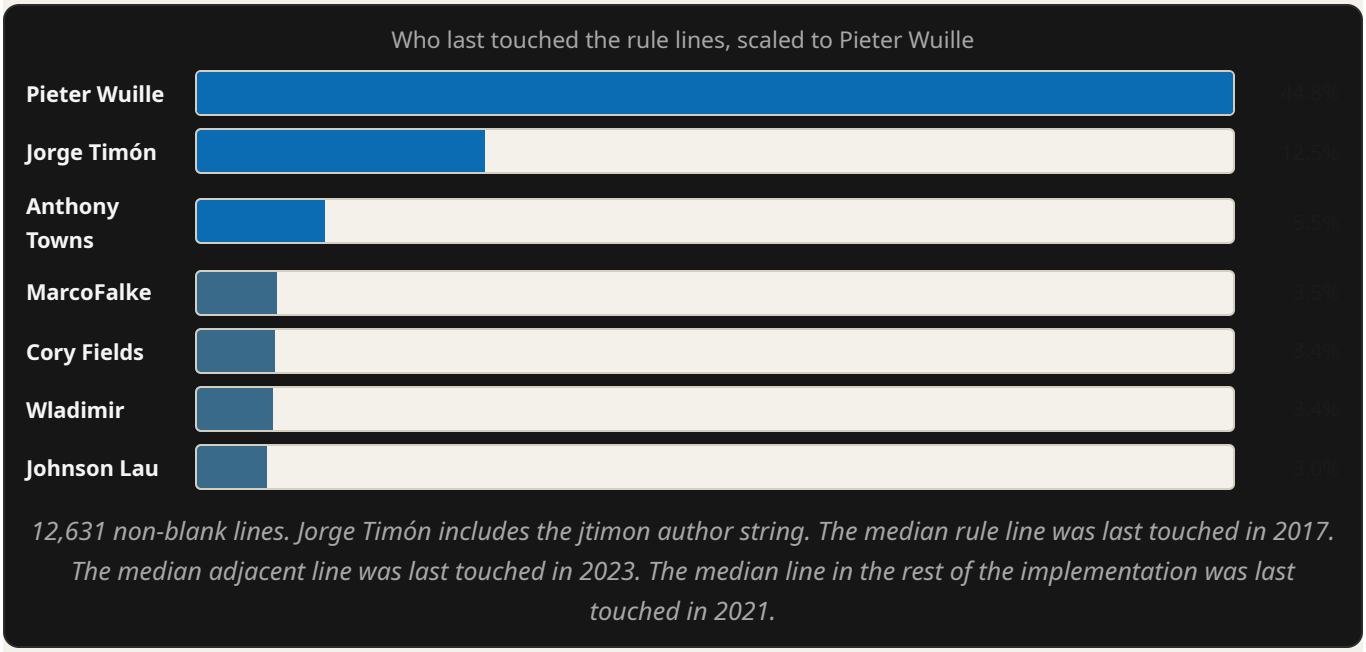
Person	Added	Deleted	Net	Commits	Deleted share
Jonas Schnelli	16,919	7,400	9,519	454	44%

fanquake's code-file net is 441 because this filter skips build scripts. His merge share is a different measurement, in *Who Controls Bitcoin*. Line authorship and merge authority are not the same ranking. Pieter Wuille leads additions. The merge log does not.

On a coarser directory sort of the same git lines, tests gained the most net (154,628 of 283,483 added). Validation kept 31,065 of 107,785. The GUI kept 23,005 of 84,571. Crypto kept 8,032 of 27,075. That sort counts all of `src/node` as P2P, so it does not match the 16,428 current P2P bar.

IX. Who last touched what remains

`git blame -w` on the 744 implementation files. Each non-blank line goes to the last commit that changed it. `validation.cpp` is split on the same function boundaries as section IV. Comments count, so these totals sit above the tokei code lines: 12,631 non-blank rule lines against 10,055 code lines.



12,631 non-blank lines. Jorge Timón includes the jtimon author string. The median rule line was last touched in 2017. The median adjacent line was last touched in 2023. The median line in the rest of the implementation was last touched in 2021.

Bucket	Non-blank lines	Most of those lines	Share	Median year
Rules	12,631	Pieter Wuille	44.8%	2017
Adjacent	26,144	Pieter Wuille	25.9%	2023
The rest	141,672	Ava Chow	11.3%	2021

Across all 180,447 non-blank implementation lines, Pieter Wuille last touched 14.4%, Ava Chow 9.1%, MarcoFalke 7.8%, and Wladimir J. van der Laan 6.4%. Those four, plus TheCharlatan, Russell Yanofsky, Hennadii Stepanov, and Vasil Dimov, last touched 50.8%. glozow is second on the adjacent lines, at 7.5%, and tenth overall.

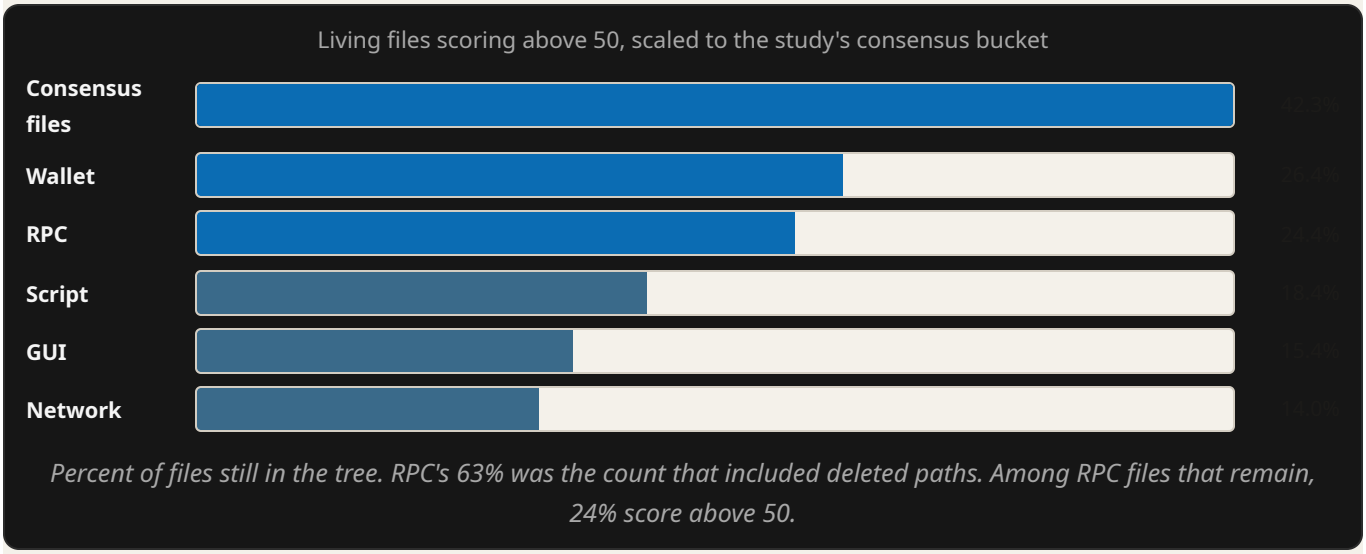
The rule lines are older than the code around them, and one person last touched almost half of them. The additions table in section VIII is a different question. Satoshi's author string leads a large share of historical additions and is absent from the lines that remain.

X. Patch debt

The January 2026 debt study scores a file from how often it is edited in small diffs and how long it has gone without a large rewrite. This is the same formula, rerun on **17,072** merged pull requests from 3 January 2011 through 12 October 2026. It is a churn score. It is not a bug count.

Counting every path that history ever touched, including files that are gone, **28.2%** score above 50 and the patch-to-refactor ratio is **3.0**. That reproduces the January headline (26.8% and 3.0). RPC is 63% of that wider count.

Restricted to files still in this tree, **16.3%** of files score above 50, and those files hold **10.2%** of current code lines. High-scoring files are smaller than average. Many of the old high scores were paths that no longer exist (`main.cpp`, `ui.cpp`). Among living files the ratio is **3.6** patches per refactor. Wallet is **6.9** to 1. The study's own consensus bucket, 26 files, is patched once per refactor, and 42% of those files still score above 50. That bucket is not the 10,055 rule lines.



The highest living source files on this score are `src/index/base.cpp` (80, 370 lines), `src/rpc/util.cpp` (79.4, 1,222 lines), `src/wallet/walletdb.h` (79.3), `src/interfaces/chain.h` (77.9), and `src/validationinterface.cpp` (72.3).

XI. What the count is for

The rules that have to be specified, audited, and matched by any other implementation are 10,055 code lines, plus a signature library that is larger than they are. The program operators download is the other 93% of the implementation, the tests, and the bundled trees. Treating a release as "the consensus rules" collapses those piles.

Don't Trust, Verify is about that collapse at the review layer. The first independent audit of Bitcoin Core, Quarkslab in September 2025, scoped P2P, mempool, block validation, and chain state. Those are mostly the adjacent lines in section V, plus some of the checks. The interpreter, the hash implementations, and the type of the transaction are the rest of the 10,055, and secp256k1 is another 63,500.

Who Controls Bitcoin measures who merges. This article measures who wrote lines and who last touched the lines that remain. The rankings differ. Merge concentration answers who can land a change. The blame in section IX answers whose text is still in the rule files. Both can be true, and neither is a substitute for the other.

Bitcoin Core: The Biggest Fallacies argues the client is a monolith because the wallet, the network, and the rules ship together. The line count is the size of that claim. Seven percent is the rules. Eighty percent is the rest of the same binary.

What Bitcoin's Stalled Proposals Tell You uses the authorship Gini from the governance research. That Gini is on pull-request authorship. The added-line shares in section VIII are a third cut. Top-five line additions at 39.7% and a historical authorship Gini near 0.85 are the same shape at two resolutions. They are not the same number.

The cost of storing the chain this program validates is a separate account, in *Full Cost of Running a Bitcoin Node*.

XII. Method

Snapshot. `/mnt/data/bitcoin/core` at `10ca73c02cbff59f2134c0c7da3b8d0a7e727475`, 11 May 2026, "Merge bitcoin/bitcoin#34580". Not shallow. 48,864 commits, 16,463 of them merges. origin/master on 30 October 2026 was 2,407 commits ahead. The tree was not checked out to that tip. Line counts are the May tree. Authorship is history through that commit.

Code lines. tokei 14.0.0. A file is in one bucket. Implementation is `src/` excluding vendored prefixes (`secp256k1`, `leveldb`, `crc32c`, `minisketch`, `univalue`, `libmultiprocess`, `ctaes`), Qt locale, tests, fuzz, bench, markdown, and `CMakeLists.txt`. The 19-way system split uses explicit file lists for `src/node`, because that directory is not all P2P.

The three-way cut. Rule files are the SHA-256 and RIPEMD-160 sources, `hash.cpp` and `hash.h`, the script interpreter and script data, `pubkey`, `arith_uint256`, `uint256`, `src/primitives`, `src/consensus`, proof of work, versionbits, deployments, and signet. `validation.cpp` is split by function start, comment-stripped, then scaled from 4,166 raw code-like lines to tokei's 4,230, with the rounding put on the consensus part. Adjacent is chainstate and storage, policy and mempool, indexes, and mining templates, including the matching parts of `validation.cpp`. SHA-1, SHA-3, and SHA-512 stay in the rest.

Tests. The named rule unit and fuzz files are the earlier filename list (4,440) plus integer, hash, script, and block tests that name a rule type (1,346). Functional rule tests are an explicit basename list: `feature_block`, `feature_taproot`, `feature_segwit`, `feature_cltv`, `feature_csv_activation`, `feature_dersig`, `feature_nulldummy`, `feature_bip68_sequence`, `feature_signet`, `feature_versionbits_warning`, `p2p_segwit`, `p2p_invalid_block`, `p2p_invalid_tx`, `p2p_mutated_blocks`. Adjacent adds an explicit list of mining, assumeutxo, pruning, index, fee, package, and chainstate tests to the mempool and policy filename prefix. A test outside those lists stays in the rest. That undercounts a functional test that exercises the interpreter under another name.

History. `git log HEAD --no-merges --numstat`. Renames of the form `{old => new}` are rewritten to the new path. Identity merges achow101 emails, hebasto in the email, and otherwise the dominant display name, casefolded, plus the published aliases (Wladimir's three spellings, MarcoFalke's, Ava Chow's, glozow and Gloria Zhao, fanquake and Michael Ford, jtimon and Jorge Timón). People who share only a first name stay separate. Ryan Ofsky and Russell Yanofsky are different people.

Blame. `git blame -w --line-porcelain` on each implementation file. Whitespace-only changes do not move the line. Blank lines are dropped. The same alias list as the history.

Debt. `scripts/analysis/code_turnover_analysis.py` in the Bitcoin Governance Research repository, run on `data/github/prs_raw.jsonl` (25,506 pull requests, 17,072 merged with a date). The score weights patch frequency, time since a large rewrite, age of files the script labels consensus, and modifications per estimated line. Change type is a size heuristic: a refactor is a change over 30% of the estimated file or over 200 lines. The script's subsystem labels are filename patterns. They are not the three-way cut in section IV.

Sources

- Bitcoin Core commit `10ca73c02cbff59f2134c0c7da3b8d0a7e727475`, 11 May 2026
- tokei 14.0.0, file-level report on that tree
- Bitcoin Governance Research, technical debt analysis, formula rerun 30 October 2026 on the pull-request dump through 12 October 2026
- *Who Controls Bitcoin*, *The Vertical Layer Problem*, *Why Bitcoin Needs a Specification That Humans Can Read*, *Don't Trust, Verify*, *Bitcoin Core: The Biggest Fallacies*, *What Bitcoin's Stalled Proposals Tell You*, *Full Cost of Running a Bitcoin Node*

SECURESOVEREIGN

Markdown is the canonical source: https://secsov.com/articles/the-rules-are-7-percent/index.md

Formatted snapshot of https://secsov.com/articles/the-rules-are-7-percent

Josh · Secure Sovereign